



TRABALHO DE CONCLUSÃO DE CURSO

Heurísticas de Encaixe para o Coupled-Task Scheduling Problem

Propostas de restrições adicionais para minimização do makespan

Pedro Canellas Innecco

Kelwyn Filipi Oliveira Reis de Souza

Orientador: André Renato Villela da Silva



Tarefas acopladas por um intervalo exato

É um problema de escalonamento: dadas N tarefas, todas devem ser escalonadas em uma única máquina.



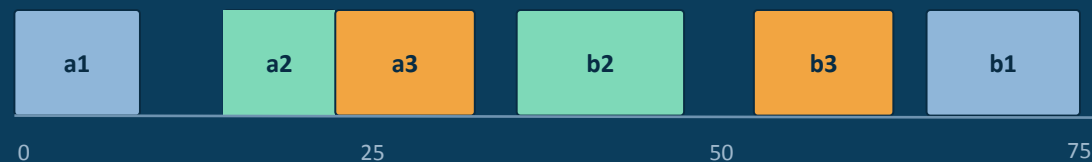
Origem: Modelagem de sistemas de radar de pulso (Shapiro, 1980)

Cada tarefa i é composta por duas partes, a_i e b_i , separadas por um intervalo fixo l_i — como um sinal de radar emitido, que aguarda o eco antes de ser processado.

- **Ordem fixa:** a_i sempre antes de b_i , com intervalo l_i exato entre elas.
- **Independência:** Não há precedência entre tarefas diferentes.
- **Não preempção:** Uma vez iniciada, cada parte deve terminar sem interrupções.
- **Objetivo:** Minimizar o makespan — o instante de término da última tarefa.

Exemplo: escalonamento de 3 tarefas

makespan = 75 unidades de tempo



O grande intervalo da tarefa 1 ($l_1 = 60$) é aproveitado para encaixar as tarefas 2 e 3, reduzindo o tempo ocioso da máquina.

Um problema com aplicações reais



Defesa

Controle de sensores para guiar torpedos submarinos (Simonin et al., 2011).



Indústria química

Processos com tempos de espera exatos entre etapas de reação (Ageev & Kononov, 2007).



Saúde

Agendamento de consultas de quimioterapia seguindo protocolo médico (Condotta & Shakhlevich, 2014).



Classificado como NP-Difícil. À medida que o número de tarefas cresce, encontrar a solução ótima exige esforço computacional inviável — daí a busca por heurísticas eficazes.

Do exato ao heurístico: interleaving e nesting



Métodos exatos

Garantem a solução ótima, mas o esforço computacional cresce de forma inviável — o CTSP é classificado como NP-Difícil.



Intercalação (interleaving)

As partes finais seguem a mesma ordem cronológica das partes iniciais — se a tarefa 1 começa antes de 2, a parte final de 1 também termina antes.



Aninhamento (nesting)

Uma tarefa inteira é encaixada dentro do tempo ocioso de uma tarefa maior, invertendo a ordem.



Temos como base a formulação de Silva (2025). É essa modelagem, apresentada a seguir, que serve de base de comparação para as heurísticas de encaixe propostas neste trabalho.

Formulação base: CP de Silva (2025)

Modelagem em Programação por Restrições (CP), usada como base de comparação. Utiliza variáveis do tipo intervalo (CP Optimizer) — com início, fim e duração fixa — em vez das variáveis numéricas típicas da Programação Inteira Mista.

minimize $\max(\text{end}(b_i))$ — o makespan é o maior instante de término entre todas as partes finais b_i

(2) $\text{start}(b_i) = \text{end}(a_i) + l_i$ Acoplamento exato entre as duas partes de cada tarefa

(3) $\text{presence}(a_i, b_i) = \text{true}$ Execução obrigatória de todas as tarefas

(4) $\text{noOverlap}(i, i')$ Não sobreposição: a máquina executa uma parte por vez

(5)-(6) $\text{length}(a_i) = A_i$; $\text{length}(b_i) = B_i$ Durações fixas das partes inicial e final

A não preempção é garantida automaticamente pela própria natureza das variáveis de intervalo — dispensando restrição explícita.

Superar a formulação base com Heurísticas de Encaixe (Nesting)

Ideia central: se partes de outras tarefas cabem no intervalo ocioso de uma tarefa maior (aninhamento), é possível buscar sistematicamente a configuração de encaixe que minimize o tempo ocioso restante — reduzindo o makespan. O escalonamento encontrado é imposto ao solver como restrições adicionais.

Uma intuição comum às evoluções: desde V1 a V1.4, cada variante testada busca adiantar ao solver um raciocínio de posicionamento que parece logicamente eficaz — entregando-lhe, via restrições adicionais, um ponto de partida forte em vez de deixá-lo redescobrir do zero a estrutura do encaixe.

Evolução experimental

1

V1 — Blocos Âncora

Aninhamento recursivo a partir da maior tarefa

2

V2 — Correntes

Elos entre partes de tarefas distintas

3

V1.4 — Final

Remoção e recolocação sistemática de tarefas

V1 — Heurística de Blocos Âncora



Aninhamento recursivo

A tarefa de maior intervalo da instância é a âncora. Recursivamente, encaixa-se a maior tarefa candidata que caiba no espaço remanescente, até esgotar as opções compatíveis.

Exemplo V1



Todo o bloco encaixado na âncora é imposto ao solver como restrição de posicionamento.

Variantes testadas

V1.1

Agrupa múltiplas tarefas na âncora, minimizando o tempo ocioso remanescente.

V1.2

Divide o tempo de processamento igualmente entre a lógica V1 e a V1.1.

V1.3

Restringe a busca da V1.1 às tarefas com os 20% maiores intervalos.

Formulação matemática — V1 e V1.1

Restrições adicionais impostas ao solver a partir do bloco de encaixe encontrado pela heurística

V1

$$(7) \quad \text{start}(a_{i+1}) \geq \text{end}(a_i)$$

Cada parte inicial do bloco só começa após o término da parte inicial anterior, respeitando a ordem de encaixe.

$$(8) \quad \text{end}(b_{i+1}) \leq \text{start}(b_i)$$

Cada parte final do bloco termina antes do início da parte final anterior — a ordem das partes b é invertida em relação às partes a.

V1.1

$$(9) \quad \text{start}(a_i) \geq \text{end}(a_{\text{âncora}(1)})$$

As partes iniciais das tarefas agrupadas só podem começar após o término da primeira parte da âncora.

$$(10) \quad \text{end}(b_i) \leq \text{start}(a_{\text{âncora}(1)})$$

As partes finais das tarefas agrupadas terminam antes do início da parte inicial da âncora, liberando o intervalo ocioso.

V2 — Modelos de Correntes



Elos em corrente entre tarefas

O término da primeira parte de uma tarefa é acoplado ao início da segunda parte de outra, formando um fluxo contínuo sem espaços ociosos.

Variantes testadas

V2.1

Testa diferentes percentuais de tarefas submetidas ao acoplamento em corrente.

V2.2 · Tripleto

Insere uma terceira tarefa cujo intervalo aninha o elo colado das duas primeiras.

V2.3

Relaxa o acoplamento, permitindo pequenas lacunas (gaps) entre as tarefas.

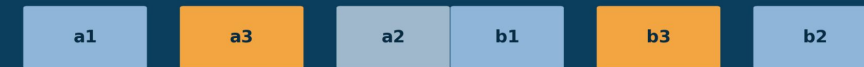
Variantes V2 — exemplos

V2 — elo básico



a2 e b1 são coladas — blocos separados, mas sem intervalo (idle time) entre eles.

V2.2 — aninhamento tripleto



a3 aninha o par colado a2/b1; b3 fecha o aninhamento tripleto.

V2.3 — lacunas (gaps) relaxadas



pequenos gaps são permitidos entre a2 e b1, relaxando o acoplamento exato.

Formulação matemática — V2 e V2.2

Restrições adicionais para os elos em corrente entre partes de tarefas distintas

V2

(11) $\text{end}(aL_2) = \text{start}(bL_1)$

O término da primeira parte da tarefa L_2 é acoplado exatamente ao início da segunda parte da tarefa L_1 , formando um elo colado sem intervalo.

V2.2

(12) $\text{end}(aL_1) \leq \text{start}(aL_2) \leq \text{start}(aL_3)$

Ordena as partes iniciais das três tarefas envolvidas no encaixe triplete.

(13) $\text{end}(bL_1) \leq \text{start}(bL_2) \leq \text{start}(bL_3)$

Ordena de forma equivalente as partes finais das três tarefas.

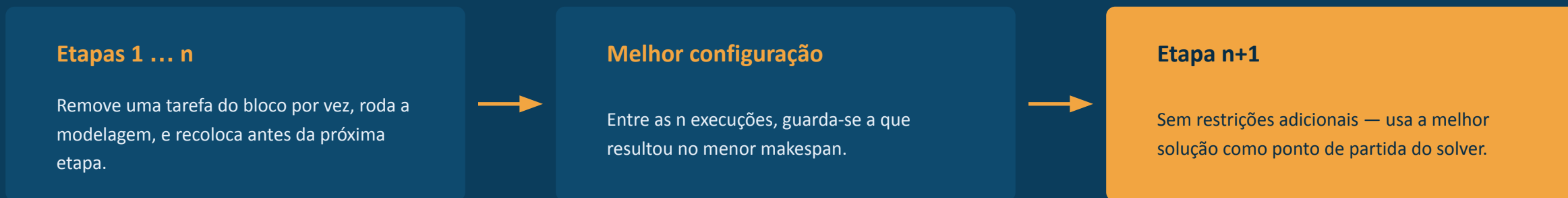
(14) $\text{end}(aL_3) = \text{start}(bL_1)$

A terceira tarefa (L_3) fecha o elo, com sua parte inicial aninhando o acoplamento colado entre L_1 e L_2 .

V2.3 — mesma lógica da V2.2 (restrições 12 e 13), porém sem a restrição (14): pequenas lacunas (gaps) passam a ser permitidas entre os elos.

V1.4 — A Heurística Final

Remoção e recolocação sistemática de cada tarefa do bloco encaixado, uma por vez, usando a melhor configuração encontrada como solução inicial do solver.



As etapas de remoção são exemplificadas a seguir

Cada uma das n etapas remove temporariamente uma tarefa do bloco encaixado pela V1 e roda novamente a modelagem. Os slides seguintes ilustram, passo a passo, cenários de exemplo.

V1.4 — Exemplo: remoção de a_3 e b_3

Primeira das n etapas: a tarefa 3 (o par a_3/b_3) é temporariamente removida do bloco encaixado pela V1 e a modelagem é executada sem ela.

Etapa 1 — remoção de a_3 e b_3



Com a_3 e b_3 fora do bloco, o solver reotimiza o posicionamento das tarefas remanescentes dentro do intervalo da âncora antes da tarefa 3 ser recolocada.

V1.4 — Exemplo: remoção de a_2 e b_2

Segunda etapa: a_2 e b_2 são removidas do bloco, e a modelagem roda novamente com as tarefas restantes.

Etapa 2 — remoção de a_2 e b_2



A remoção da tarefa 2 testa se abrir esse espaço permite um posicionamento mais eficiente das demais tarefas antes da mesma retornar ao bloco.

V1.4 — Exemplo: remoção de a_1 e b_1

Terceira etapa: a_1 e b_1 (a própria âncora) são removidas, e o solver reotimiza o restante do bloco sem elas.

Etapa 3 — remoção de a_1 e b_1

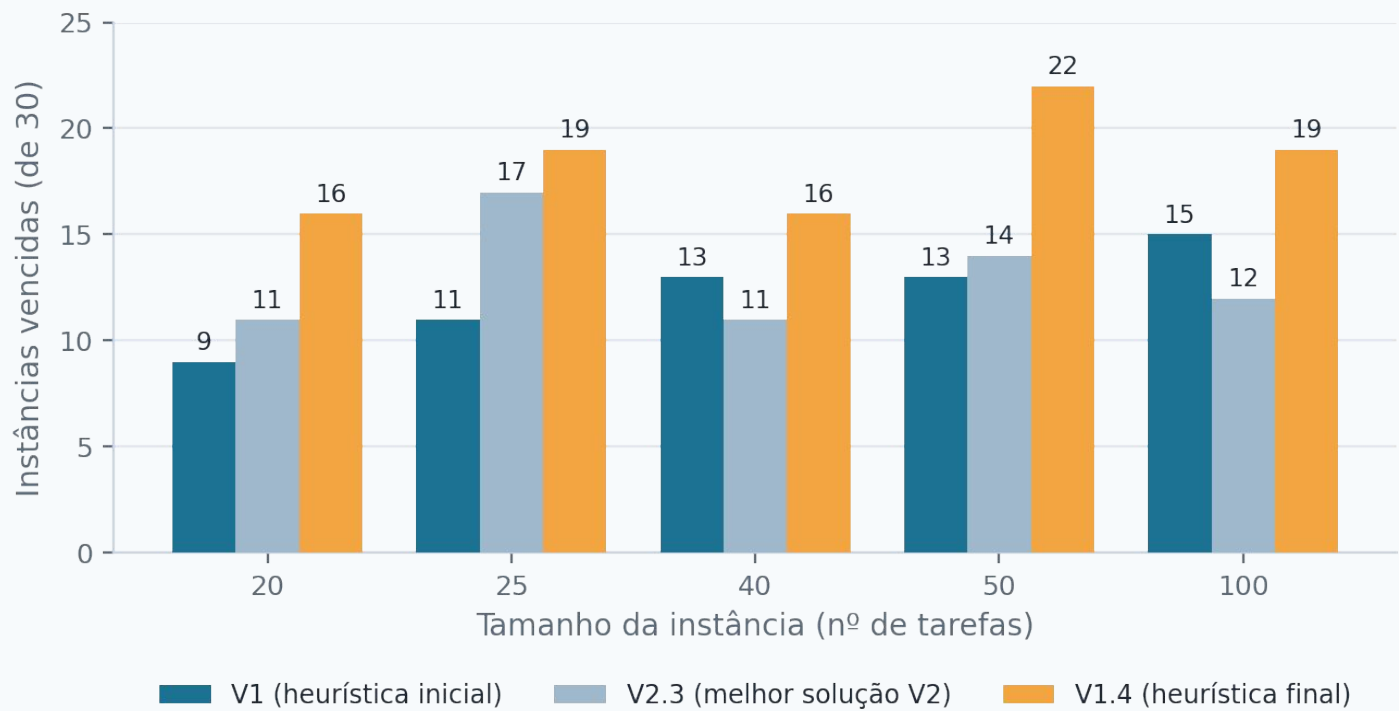


Ao final das n etapas, guarda-se a melhor configuração encontrada entre todas as remoções — usada como solução inicial na etapa $n+1$, sem restrições adicionais.

RESULTADOS

V1.4 supera a modelagem base em 92 das 150 instâncias

CP Optimizer	Limite de tempo	Instâncias	Tamanhos
22.1.1.0	600s / instância	150 (30 × 5 tamanhos)	20, 25, 40, 50, 100



92 / 150

instâncias vencidas pela V1.4 sobre a formulação de Silva (2025)

Ganhos consistentes em todos os tamanhos

A V1.4 supera as demais em todos os grupos, confirmando os ganhos sucessivos da remoção/recolocação sobre o aninhamento simples e sobre o acoplamento rígido.

Heurísticas de encaixe como boas geradoras de soluções para o CTSP

- **Remoção sistemática funciona** — A V1.4 (remoção e recolocação de tarefas) foi a estratégia mais eficaz, vencendo a base em 92 das 150 instâncias.
- **Restringir demais prejudica** — Limitar a busca às tarefas com maiores intervalos (V1.3) piora os resultados, sobretudo em instâncias grandes.
- **Acoplamento rígido não compensa** — O posicionamento fixo entre tarefas (V2) frequentemente aumenta o makespan por conflito com tarefas já posicionadas.



Trabalhos futuros

- Instâncias maiores ($N > 100$)
- Múltiplos blocos-âncora simultâneos na mesma instância
- Código-fonte público para reprodutibilidade



github.com/PCanellas/tcc_si_2026.1

Obrigado.